

파이썬 프로그래밍과 시간복잡도

Python Programming and Time Complexity

2주차 (2) · 자료구조

2026-2 학기 · 김지언

목차

01 파이썬 프로그래밍

- 파이썬 프로그램의 구조와 실행 방식
- 식별자와 데이터 타입
- 제어문 : 조건문과 반복문
- 함수 · 메서드 호출

02 객체지향 프로그래밍

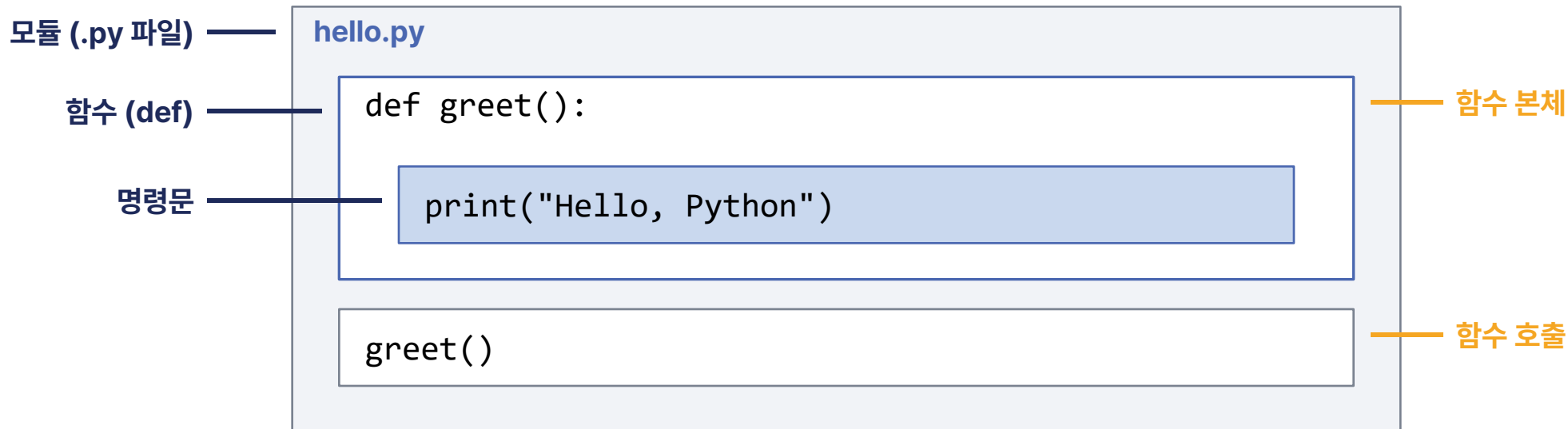
- 클래스 · 인스턴스 · 속성 · 메서드
- 캡슐화 · 상속성 · 다형성
- 객체지향 언어로서의 파이썬

03 시간복잡도

- 시간복잡도란? 왜 추상화가 필요한가
- Big-O 표기법과 계산 예제
- 정렬 비교 : 선택 정렬 vs 병합 정렬
- 자료구조 선택으로 복잡도 개선하기
- 연습문제 · Big-O 비교표 (부록)

파이썬 프로그래밍

파이썬 프로그램의 구조 – hello.py 프로그램 분석



함수 (def)

- 수행할 명령문의 묶음
- 함수 이름과 본체로 구성
- 본체에 한 개 이상의 명령문 포함
- 들여쓰기로 본체 범위를 구분

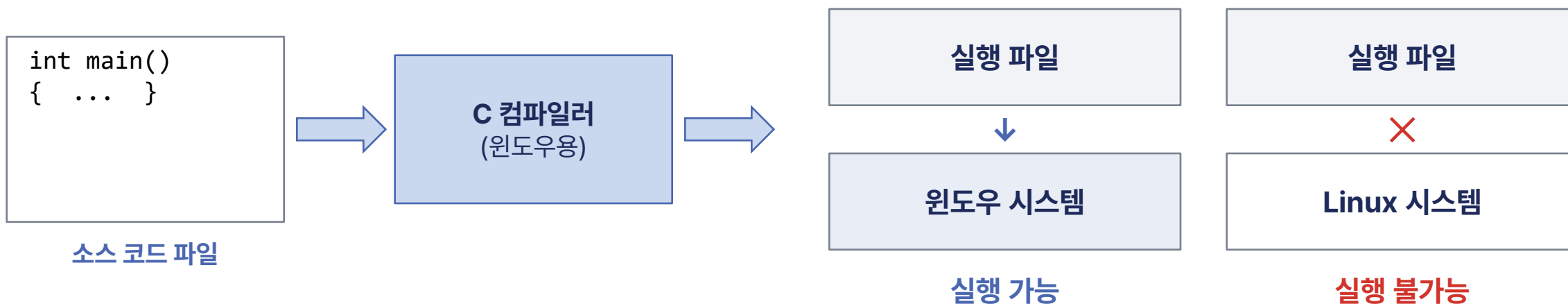
모듈 (.py 파일)

- 파이썬 프로그램을 구성하는 단위
- 파일 이름이 곧 모듈 이름
- 최상위 명령문을 위에서 아래로 실행
- `if __name__ == "__main__":` – 실행 시작 지점

파이썬 프로그래밍

컴파일 언어의 실행 파일은 운영체제에 종속

- **C, C++ 같은 컴파일 언어로 만든 실행 파일은 같은 운영체제에서만 실행 가능**
 - ▶ 다른 시스템에서 실행하려면 새로 컴파일하여 실행 파일을 다시 만들어야 하는 문제 발생 (예. 1:1 언어 번역기)
 - ▶ 컴파일된 결과를 다른 시스템에서도 그대로 실행할 수 있는 새로운 방법 필요 (예. 국제 공통어 번역기)

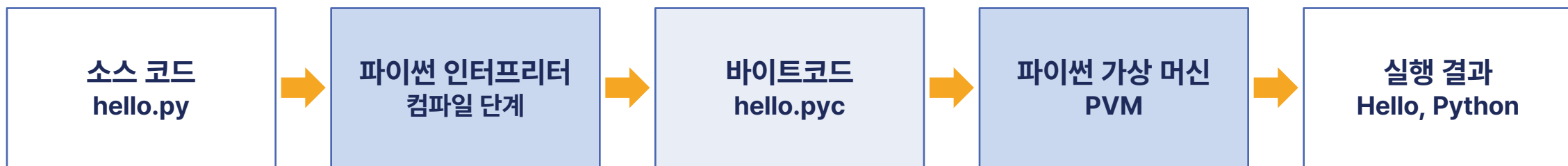


해결책 : 소스 코드를 운영체제와 무관한 중간 형태로 바꾸고, 시스템마다 설치된 실행기(인터프리터)가 번역해 실행

파이썬 프로그래밍

프로그램 실행 과정 – 파이썬 인터프리터의 실행 방식

- 파이썬은 소스 코드를 바이트코드로 변환하고, 각 운영체제에 설치된 파이썬 가상 머신이 해석하여 실행
 - ▶ 실행 파일을 만들지 않으므로 운영체제마다 다시 컴파일할 필요가 없음
 - ▶ 소스 코드(.py)만 있으면 파이썬이 설치된 모든 시스템에서 그대로 실행 가능



운영체제와 무관한 중간 코드 생성

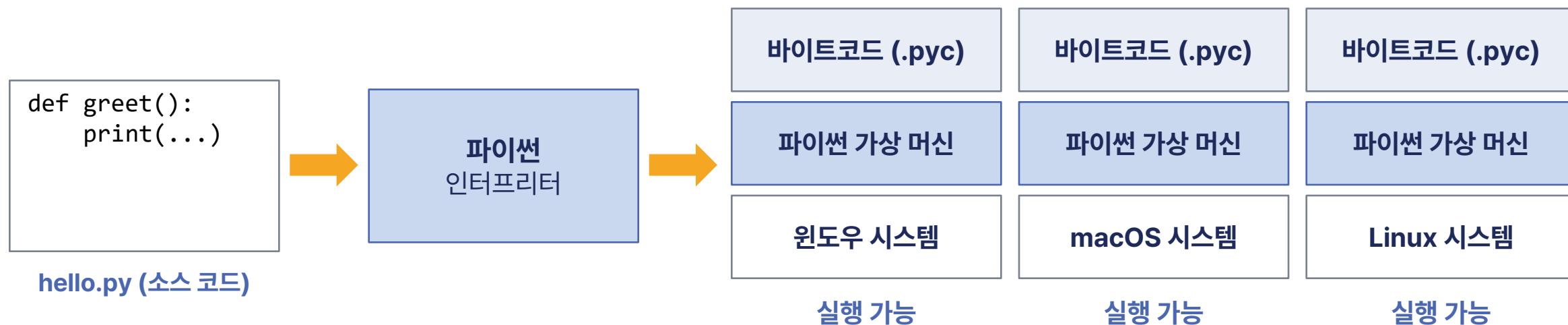
시스템마다 설치된 PVM 이 해석·실행

Write Once, Run Anywhere!

한 번 작성한 파이썬 코드를 윈도우 · macOS · Linux 에서 수정 없이 실행

파이썬 프로그래밍

파이썬 소스 프로그램의 실행 방식



- 소스 코드(.py)는 그대로 두고, 각 시스템에 설치된 파이썬이 번역하여 실행
- 운영체제별로 다시 컴파일하거나 실행 파일을 따로 만들 필요가 없음
- 실행할 때마다 해석하므로 속도는 컴파일 언어보다 느리지만, 이식성과 개발 속도가 우수

파이썬 식별자

파이썬 프로그램을 구성하는 단어

- 키워드 (예약어) – 파이썬이 문법으로 미리 정해 둔 단어
- 키워드를 제외한 나머지 단어
 - ▶ 클래스, 함수, 변수 등의 이름(식별자)으로 사용

[파이썬의 키워드]

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

참고 : 상수 값을 표현하는 True, False, None 도 키워드에 포함되며, 식별자로 사용할 수 없다.
(keyword.kwlist 로 전체 목록 확인 가능)

파이썬 식별자

파이썬 식별자를 만드는 규칙

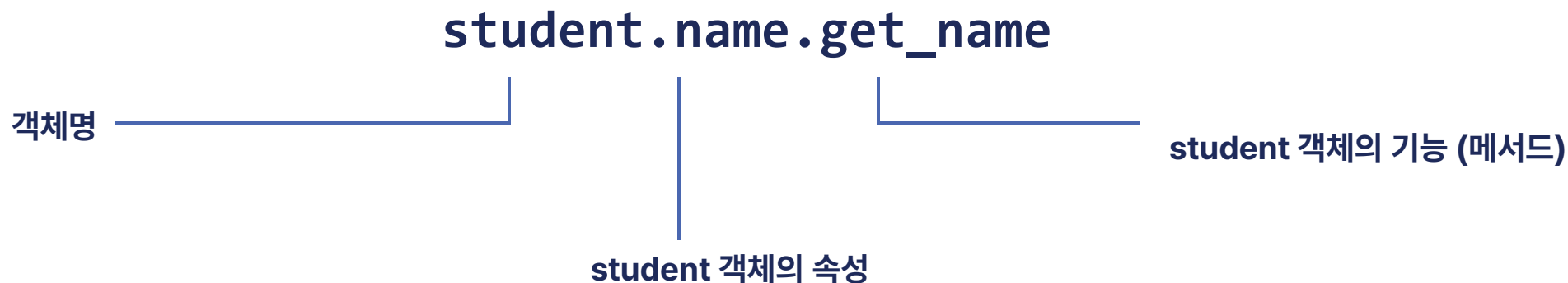
- 하나 이상의 문자로 이루어져야 한다.
- 첫 번째 문자는 영문자 또는 _ 여야 한다. (숫자로 시작할 수 없다.)
- _ 이외의 특수문자는 사용할 수 없다. (\$ 도 사용할 수 없다.)
- 문자 길이의 제한이 없다.
- 키워드는 식별자로 사용할 수 없다.
- 대문자와 소문자를 구별한다. (Name 과 name 은 서로 다른 식별자)

```
class Student:
    def __init__(self, name, school):
        self.name = name
        self.school = school

    def get_name(self):
        return self.name

student = Student("철수", "한림대학교")
```

여러 식별자를 함께 사용하는 경우에는 점(.)으로 연결



파이썬의 데이터 타입

변수(variable)

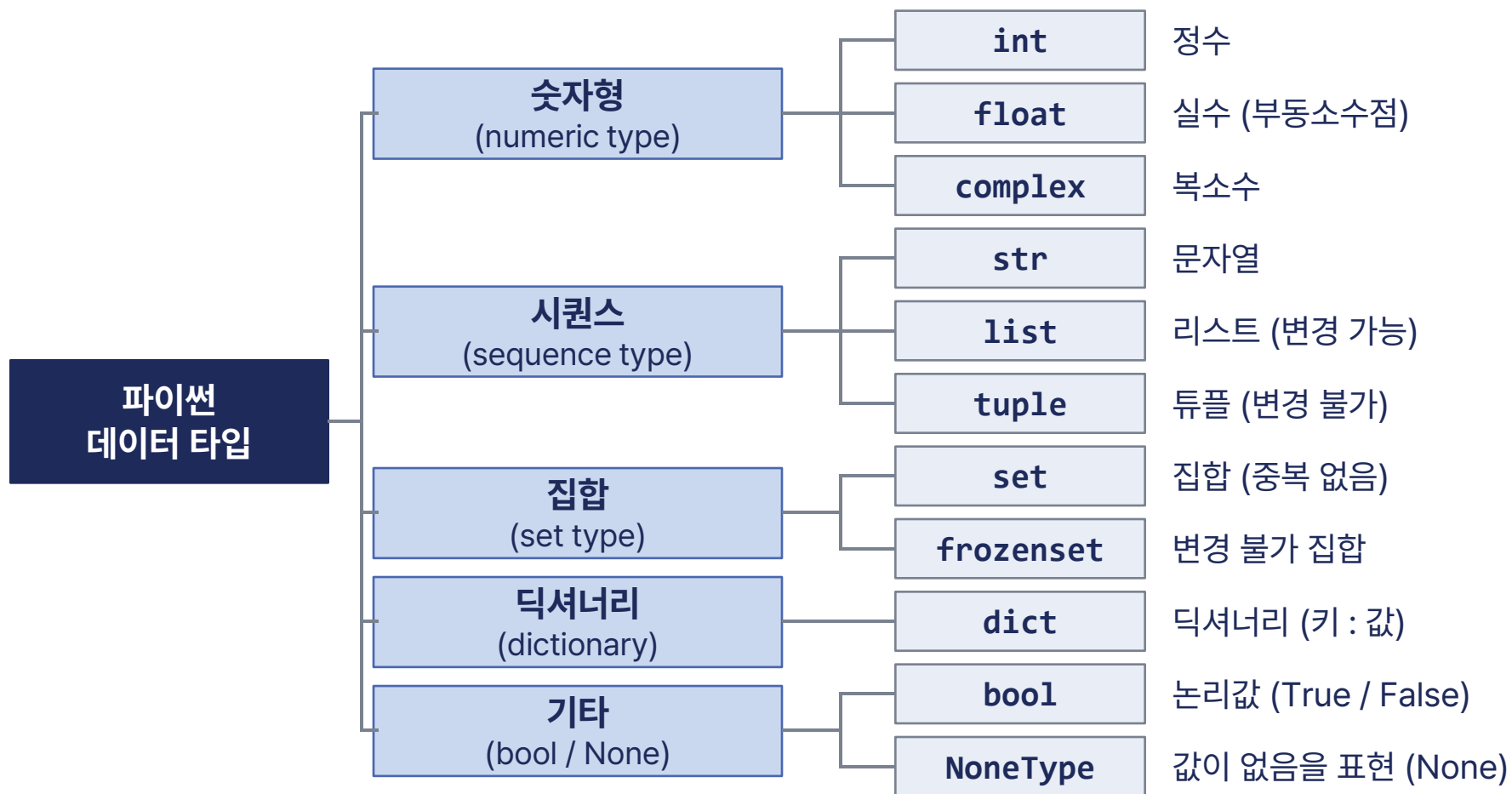
- 어떤 값을 저장하는 공간
- 변수를 따로 선언하지 않고, 값을 대입하는 순간 만들어짐
 - ▶ 데이터 타입을 적지 않아도, 대입한 값에 따라 타입이 자동으로 결정된다 (동적 타이핑)

변수 이름 sum = 1 + 2 저장할 값 (식의 결과)

- 변수를 미리 선언하지 않고, 값을 대입하면 변수가 만들어진다
- 같은 변수에 다른 타입의 값을 다시 대입할 수도 있다 : `sum = "three"`
- 현재 타입은 `type()` 함수로 확인 : `type(sum) → <class 'int'>`

파이썬의 데이터 타입

파이썬의 기본 데이터 타입



파이썬의 데이터 타입

파이썬에서 가장 자주 사용되는 자료형을 정리하면?

int

- integer — 정수형 데이터
- 5 / 3 / 17 / 223 / -87 / ...

float

- floating point number — 실수형 데이터
- 3.14 / 7.72 / -8.33 / 4.0 / ...

bool

- boolean — 참과 거짓 가운데 하나를 나타내는 자료형
- True / False

str

- string — 문자열 데이터
- "hello" / 'Python' / "3.14" / ...

파이썬의 연산자

기초 연산을 위한 파이썬 연산자 ($a = 7, b = 2$ 인 경우)

연산자	의미	예 ($a = 7, b = 2$)
$a + b$	a와 b를 더한 결과	$7 + 2 \rightarrow 9$
$a - b$	a에서 b를 뺀 결과	$7 - 2 \rightarrow 5$
$a * b$	a와 b를 곱한 결과	$7 * 2 \rightarrow 14$
a / b	a를 b로 나눈 결과 (실수)	$7 / 2 \rightarrow 3.5$
$a // b$	a를 b로 나눈 몫 (정수)	$7 // 2 \rightarrow 3$
$a \% b$	a를 b로 나눈 나머지	$7 \% 2 \rightarrow 1$
$a ** b$	a의 b제곱 (거듭제곱)	$7 ** 2 \rightarrow 49$

파이썬의 표준 출력

파이썬이 제공하는 표준 출력·입력 함수 — `print()` 와 `input()`

`print()`

- 대상을 출력하기 위해 사용하는 내장 함수
- 값 · 연산식 · 변수 · 문자열이 모두 들어갈 수 있음

코드

```
print(3 + 5)
name = "한림"
print("안녕,", name)
```

실행 결과

```
8
안녕, 한림
```

`print()` 를 쓰지 않으면 화면에 아무것도 표시되지 않습니다

`input()`

- 사용자가 입력한 값을 받아오는 내장 함수
- 입력값은 항상 문자열(str) → 계산에는 `int()` 변환 필요

코드

```
name = input("이름? ")
age = int(input("나이? "))
print(name, age + 1)
```

실행 결과

```
이름? 한림
나이? 22
한림 23
```

`int()` 없이 `age + 1` 을 하면 `TypeError` 가 발생합니다

파이썬의 표준 입력

입력되는 모든 것은 문자열이다

코드

```
print('사과 1개 가격:')  
apple = input('사과 1개 가격 입력: ')  
apple = int(apple)  
print('사과 2개 가격은', apple*2, '원 입니다.')
```

실행 결과

```
사과 1개 가격:  
사과 2개 가격은 4000 원 입니다.
```

입력 상황을 고려한 자료형 변환

- ✓ input() 으로 받은 값은 숫자 형태라도 문자열(str)
- ✓ 문자열은 int() · float() 로 변환해 계산에 사용
- ✓ 변환하지 않으면 bank * 2 는 문자열 반복이 됨
- ✓ line 2 와 line 3 은 한 줄로도 표현 가능

```
apple = int(input(  
    '사과 1개 가격 입력: '))
```

조건문과 반복문

조건문 – if 명령문

- 조건식을 검사하여

- ▶ 결과가 참(True)이면, 콜론 뒤에 들여쓰 블록의 명령문을 수행
- ▶ 결과가 거짓(False)이면, 그 블록을 수행하지 않고 건너뛴

if – 참일 때만 실행

```
age = 20
if age >= 19:
    print("성인입니다")
print("확인 완료")
# 출력 : 성인입니다
```

if ~ else – 둘 중 하나 실행

```
n = 7
if n % 2 == 0:
    print("짝수")
else:
    print("홀수")
# 출력 : 홀수
```

elif – 여러 조건을 차례로

```
score = 85
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("C")
# 출력 : B
```

핵심 : 중괄호 { } 가 없음 / 조건식 끝에 콜론(:)을 붙이고, 같은 크기로 들여쓰 줄들이 하나의 블록임

조건문과 반복문

조건문을 이용한 파이썬 프로그래밍 - 중첩 조건 다루기

문제

Python 수업과 자료구조 수업 점수를 입력받아, 평균이 60점 이상이면 합격이되 한 과목이라도 40점 미만이면 불합격을 출력하는 코드는?

생각할 점

- 합격 조건과 불합격 조건을 함께 판단
- 과락은 평균 조건 안에서 중첩 if 로
- or 는 한 과목만 미달해도 참

```
score_py = int(input())
score_ds = int(input())

avg = (score_py + score_ds)/2

if (avg>=60):
    if (score_ds <40) or (score_py<40):
        print("불합격")
    else:
        print("합격")
else:
    print("불합격")
```

입력 70 / 30 → 불합격

입력 80 / 50 → 합격

입력 50 / 40 → 불합격

조건문과 반복문

if 조건문 예제 코드

1. 짝수 · 홀수 판정

```
n = 7
if n % 2 == 0:
    print("짝수")
else:
    print("홀수")
# 홀수
```

if ~ else 로 두 갈래 중 하나만 실행

2. 두 수 중 큰 값 구하기

```
a, b = 3, 9
if a > b:
    big = a
else:
    big = b
big = a if a > b else b    # 한 줄 표현
```

조건 표현식(삼항 연산자)으로 짧게 쓸 수 있다

3. 여러 조건을 함께 검사

```
score = 85
if 80 <= score < 90:
    print("B 등급")
if score >= 60 and score % 5 == 0:
    print("합격")
# B 등급 / 합격
```

and · or · not 사용, 범위 비교는 연달아 쓰기 가능

4. 값의 존재 · 빈 값 검사

```
nums = [1, 2, 3]
if 2 in nums:
    print("있음")
if not nums:        # 빈 리스트는 False
    print("비어 있음")
# 있음
```

빈 리스트 · 0 · "" · None 은 모두 거짓으로 취급

조건문과 반복문

반복문 – for 문과 while 문

- 정해진 명령문 블록을 여러 번 반복 수행

- ▶ for 문 : 리스트 · 문자열 · range 등 시퀀스의 원소를 하나씩 꺼내면서 반복 (개수가 정해진 반복)
- ▶ while 문 : 조건식이 참인 동안 계속 반복 (끝나는 시점을 조건으로 판단)

for – 시퀀스 순회

```
fruits = ["사과", "귤", "감"]
for f in fruits:
    print(f)
# 출력 : 사과 → 귤 → 감

total = 0
for i in range(1, 6):
    total += i
print(total) # 15
```

while – 조건 반복

```
i = 1
while (i <= 10):
    print('지금까지', i, '번
        출석하셨습니다.')
    i = i+1

# 지금까지 1 번 출석하셨습니다.
# 지금까지 2 번 출석하셨습니다.
...
# 지금까지 9 번 출석하셨습니다.
# 지금까지 10 번 출석하셨습니다.
```

Break 를 이용한 반복문

```
while(True):
    num = input()
    num = int(num)

    if (num < 0):
        break
```

조건문과 반복문

반복문 예제 – 리스트 요소 출력과 반복의 기본

1. for 문 – 리스트 요소 출력

코드

```
nums = [10, 20, 30]

for n in nums:
    print(n)

print("합계 :", sum(nums))
```

실행 결과

```
10
20
30
합계 : 60
```

2. while 문 – 리스트가 빌 때까지 꺼내기

코드

```
nums = [10, 20, 30]

while nums:                # 비어 있으면 종료
    print(nums.pop())      # 맨 뒤부터 꺼냄

print(nums)
```

실행 결과

```
30
20
10
[]
```

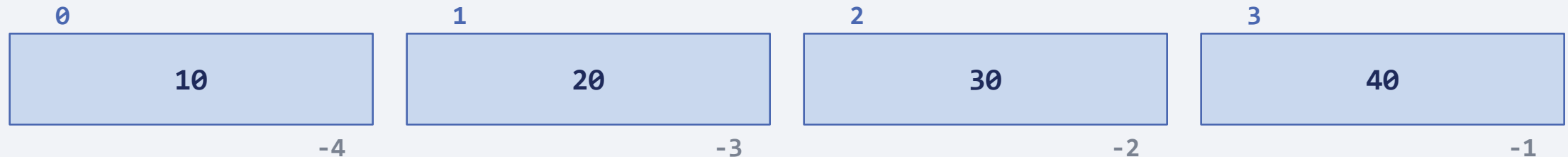
조건문과 반복문

새로운 원소를 list 에 추가하는 방법 (append, insert)

- 파이썬이 제공하는 container data type 중 하나
- 자료형과 상관 없이, 여러 개의 원소를 하나의 list 에 저장할 수 있음
- list 를 새로 만들 수도 있고, 이미 있는 list 의 원소를 수정 · 추가 · 삭제할 수도 있음
- 생성과 원소 접근에는 모두 대괄호 [] 를 사용 — append / pop 은 맨 뒤에서 O(1)

list – append / pop / 인덱싱

```
nums = [10, 20, 30]
nums.append(40)    # 맨 뒤에 추가
print(nums)        # [10, 20, 30, 40]
print(nums.pop())  # 40 (맨 뒤 꺼내기)
print(nums[0], nums[-1]) # 10 30
print(nums[0:2], len(nums)) # [10, 20] 3
```



인덱스는 0부터, 음수 인덱스는 뒤에서부터

조건문과 반복문

문자열과 리스트에 같은 방식으로 접근하기

- 문자열 = 순서
- 리스트 = 순서
- 리스트에서 index 이용해 원소에 접근한 것과 같이 문자열에서도 같은 방식으로 접근??

course_name =

s

t

r

u

c

t

u

r

e

문자열 슬라이싱

```
# 문자열 슬라이싱
```

```
course_name = 'structure'
```

```
course_name[2:4]          # 'ru'    ← 2번부터 3번까지 (4번은 포함 안 됨)
```

조건문과 반복문

문자열을 `split()` 으로 쪼개 리스트로 만들기

- `input()` 으로 받은 값은 언제나 하나의 긴 문자열
- **`split(구분자)` 는 문자열을 구분자 기준으로 잘라 `list` 로 돌려줌**
- 구분자를 생략하면 공백(스페이스 · 탭 · 줄바꿈) 기준으로 잘라 줌
- 쪼갠 뒤에는 `list` 이므로 인덱싱 · 반복문 · `len()` 을 그대로 사용 가능

문자열 - `split()` 으로 쪼개기

```
address = input('주소를 쉼표로 구분하여 입력하세요: ')
address_list = address.split(',')
print(address_list)
```

```
# 실행 결과 (입력: 강원도, 춘천시, 한림대학길 1, 공학관)
['강원도', ' 춘천시', ' 한림대학길 1', ' 공학관']
```

쉼표 뒤 공백까지 함께 들어오므로, 필요하면 `strip()` 으로 앞뒤 공백을 제거

조건문과 반복문

연속된 숫자 만들기 — range 객체와 list() 는 다르다

- `range(1, 100)` 은 리스트가 아니라 `range` 객체 — 숫자를 미리 만들지 않고 필요할 때 하나씩 꺼내 준다
- 끝 값은 포함되지 않음 — `range(1, 100)` 은 1 부터 99 까지 총 99 개
- 인덱싱 · `len( )` · `for` 는 되지만, `append( )` 같은 추가 · 수정은 불가
- 리스트로 다루고 싶으면 `list(range(...))` 로 변환해서 사용

range 객체 vs list – `type( )` 으로 확인하기

```
num_range = range(1, 100)
print(type(num_range))          # <class 'range'> ← 리스트 아님
print(num_range[98], len(num_range)) # 99 99 (인덱싱 · len 은 가능)

nums = list(range(1, 100))      # 리스트로 변환
print(type(nums))               # <class 'list'>
nums.append(100)                # 리스트는 추가 가능 (range 는 불가)
```

`range` 는 '숫자를 만들어 주는 규칙', `list` 는 '숫자를 모두 꺼내 담아 둔 상자'

조건문과 반복문

range 로 되는 것 · 안 되는 것

- 읽기 문법(인덱싱 · 슬라이싱 · len · in · for · min/max/sum)은 리스트와 동일
- 값을 수정 · 추가 · 삭제하는 문법은 리스트만 — 필요하면 list(range(...)) 로 변환

되는 것 – 읽기 (리스트와 동일)

```
r = range(1, 100)
r[0], r[-1], r[98]    # 1 99 99
r[10:20]              # range(11, 21)
len(r)                # 99
50 in r               # True
for i in r: ...
min(r), max(r), sum(r)
list(r), tuple(r)
```

안 되는 것 – 수정 (리스트만 가능)

```
r[0] = 5              # TypeError
r.append(100)          # AttributeError
r.pop()               # 없음
r.sort()              # 없음
r.reverse()           # 없음

nums = list(r)         # 변환하면
nums.append(100)       # 모두 가능
```

슬라이싱 결과도 리스트가 아니라 range — r[10:20] 은 range(11, 21)

조건문과 반복문

for 문 — list 와 range 로 반복하기

- **for 변수 in 반복할 대상 :** — 대상의 원소를 하나씩 꺼내 변수에 담아 반복
- list 를 쓰면 원소를 차례대로, range 를 쓰면 숫자를 차례대로 꺼냄
- 콜론(:) 다음 줄은 반드시 들여쓰기 — 들여쓰는 줄이 반복되는 범위

for 문 활용 – list / range

```
# for 문 활용 (list)
coffee = ['americano', 'latte', 'coldbrew']
for menu in coffee:
    print(menu, "한 잔 주세요.")
→ americano 한 잔 주세요. / latte 한 잔 주세요. / coldbrew 한 잔 주세요.
```



```
# for 문 활용 (range)
for i in range(1, 11):
    print('커피', i, "잔 주세요.")
→ 커피 1 잔 주세요. ~ 커피 10 잔 주세요. (10 번 반복)
```

`range(1, 11)` 은 1 부터 10 까지 — 끝 값 11 은 포함되지 않음

조건문과 반복문

while 문 — 조건이 참인 동안 반복 (break / continue)

- for 는 횟수가 정해진 반복, while 은 조건이 참인 동안 계속하는 반복
- 조건을 바꿔 주는 줄($i = i + 1$)이 없으면 끝나지 않는 무한 루프가 됨
- break 는 반복을 즉시 끝내고, continue 는 이번 회차만 건너뛴

while 문 활용 – break / continue

```
# while 문 활용
```

```
cup = 1
```

```
while cup <= 3:
```

```
    print('커피', cup, "잔 주세요.")
```

```
    cup = cup + 1
```

```
→ 커피 1 잔 주세요. / 커피 2 잔 주세요. / 커피 3 잔 주세요.
```

```
# break / continue
```

```
for i in range(1, 11):
```

```
    if i == 4: break          # 4 에서 반복 종료 → 1 2 3
```

```
    if i % 2 == 0: continue # 짝수는 건너뛴
```

dictionary 와 함수

dict 기초 — 과일 이름(키)으로 가격(값) 저장하고 꺼내기

- dict 는 { 키 : 값 } 쌍으로 저장 — 과일 이름을 알면 가격을 바로 찾을 수 있음
- 값을 꺼낼 때는 price['banana'] 처럼 대괄호 안에 키를 적음
- keys() 는 키만 모아 주므로 for 문과 함께 전체 과일을 훑을 수 있음

dict – 값 꺼내기 / 추가 / keys() 로 반복

```
price = {'apple': 2000, 'banana': 500, 'pineapple': 5000} # dict 만들기

print(price['banana'])      # 500   (키로 값 꺼내기)
price['melon'] = 8000       # 없는 키 → 새로 추가

for fruit in price.keys():  # 키를 하나씩 꺼내 반복
    print('오늘의 과일: ', fruit)
→ 오늘의 과일:  apple / banana / pineapple / melon
```

list 는 순서(index)로 찾고, dict 는 키(이름)로 찾는다 — keys() · values() · items()

dictionary 와 함수

기능 정의하기, 함수 — def · 인자(argument) · 반환(return)

- 사용하고자 하는 기능을 함수로 미리 정의해둔 후, 호출해서 사용
- 함수 바깥으로부터 인자(argument)를 입력받아, 결과를 반환(return)
- 사용자 정의 함수 외에도 print() · len() 같은 내장 함수가 이미 존재

함수 – 정의(def) / 호출 / 반환(return)

```
# 함수 정의 – def 함수이름(인자):  
def order(menu, count):  
    price = {'apple': 2000, 'banana': 500}  
    return price[menu] * count      # 결과를 돌려줌  
  
# 함수 호출  
total = order('apple', 3)          # 인자 2개 전달  
print(total)                       # 6000  
print(len('python'))               # 6   (내장 함수)
```

(def)는 '기능을 만들어 두는 일', 호출은 '만들어 둔 기능을 실제로 쓰는 일'

dictionary 와 함수

함수 예제 — 세 수 중 가장 큰 값 반환하기 (triple_max)

- 문제: 인자로 세 자연수를 받아, 가장 큰 수의 값을 반환하는 함수를 정의하면?
- 조건문으로 '자기 자신이 나머지 둘보다 크거나 같은지' 차례대로 확인
- return 을 만나면 함수는 그 자리에서 끝나므로 아래 조건은 검사하지 않음

함수 예제 – triple_max(a, b, c)

```
def triple_max(a, b, c):  
    if (a >= b) and (a >= c):  
        return a  
    if (b >= a) and (b >= c):  
        return b  
    if (c >= a) and (c >= b):  
        return c
```

```
print(triple_max(1, 7, 50))    # 7
```

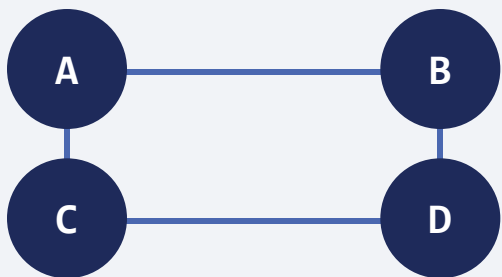
dictionary 와 함수

자료구조에 자주 쓰는 dict — 그래프(인접 리스트) 표현

- 파이썬이 제공하는 container data type 중 하나 — 키(key) 와 값(value) 의 쌍으로 저장
- 자료형과 상관 없이, 키에 값을 연결해 저장할 수 있음 (키는 문자열 · 숫자 같은 불변 자료형)**
- 키로 값을 바로 찾고, 키 단위로 값을 수정 · 추가 · 삭제할 수 있음
- 생성에는 중괄호 {}, 원소 접근에는 대괄호 [] 를 사용

dict - 그래프(인접 리스트) 표현

```
graph = {"A": ["B", "C"],      # 선으로 이어진 이웃을 모두 적는다
         "B": ["A", "D"],
         "C": ["A", "D"],
         "D": ["B", "C"]}
print(graph["A"])              # ['B', 'C']
for v in graph["C"]: print(v)  # A  D
```



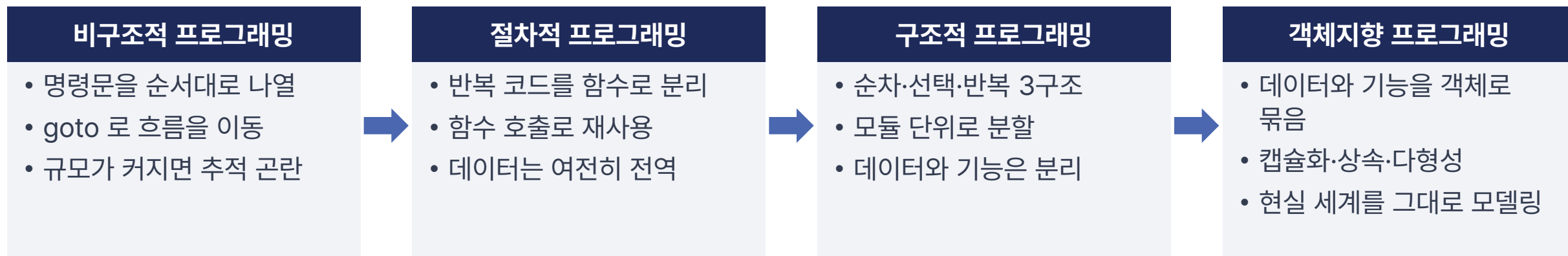
- 그래프 = { 노드 : 이웃 노드 리스트 } · 평균 $O(1)$ 조회
- 무방향 그래프는 간선 하나를 양쪽에 두 번 적는다**
- A-C 는 A 의 이웃에도, C 의 이웃에도 들어간다

객체지향 프로그래밍

프로그래밍 기법의 발달 - 복잡성을 다루는 방법의 진화

• 프로그램의 규모가 커지면서 코드를 관리하는 방법도 함께 발전

- ▶ 명령을 순서대로 나열하는 방식은 규모가 커지면 실행 흐름을 추적하기 어려움 (예: 스파게티 코드)
- ▶ 기능을 분리하는 단계를 거쳐, 데이터와 기능을 하나로 묶는 방향으로 발전 (예: 부품 단위 조립)



발전의 방향 : 명령의 나열 → 기능의 분리 → 데이터와 기능의 결합.
규모가 커질수록 "무엇을 하나로 묶을 것인가"가 핵심 문제가 됨

객체지향 프로그래밍

절차적 방식과 객체지향 방식 – 같은 문제, 다른 구조

절차적 방식 – 데이터와 기능이 분리

```
scores = [학생, 학번]    # 전역 데이터
```

```
def add(s): ...  
def average(): ...  
def report(): ...
```

전역 데이터를 여러 함수가 함께 수정

절차적 방식의 한계

- 전역 데이터를 누가 바꿨는지 추적이 어려움
- 기능을 수정할 때 영향 범위 파악이 힘들
- 비슷한 코드를 매번 새로 작성
- 규모가 커질수록 유지보수 비용 급증

객체지향 방식 – 데이터와 기능을 하나로

```
class ScoreBook:  
    def __init__(self):  
        self.scores = []  
    def add(self, s): ...  
    def average(self): ...
```

데이터와 메서드를 객체 안에 캡슐화

객체지향 방식의 이점

- 접근 범위를 제한해 데이터를 보호 (캡슐화)
- 기존 클래스를 물려받아 확장 (상속)
- 같은 호출로 서로 다른 동작 수행 (다형성)
- 객체 단위로 나눠 협업과 재사용에 유리

객체지향 프로그래밍

객체란 무엇인가 - 클래스 · 객체(인스턴스) · 속성

- 객체 = 데이터(속성) + 그 데이터를 다루는 기능(메서드)을 하나로 묶은 단위

- ▶ 클래스는 객체를 만드는 설계도 → 설계도로 실제로 만들어진 것이 인스턴스(instance) = 객체(Object)
- ▶ 클래스 하나(Dog)로 인스턴스를 여러 개(my_dog, your_dog) 만들 수 있다

class Dog: 클래스 (설계도 · 추상)

속성 (데이터)

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

기능 (메서드)

```
def bark(self):
def run(self):
```

인스턴스화
(instantiation)



인스턴스 (instance) = 오브젝트 - 메모리에 실제로 만들어진 실체

```
my_dog = Dog("Hodu", 5)
name = "Hodu" · age = 5 · bark() 사용 가능
```

```
your_dog = Dog("Coco", 2)
name = "Coco" · age = 2 · 같은 기능, 다른 값
```

클래스는 설계도(추상), 인스턴스는 그 설계도로 만든 실체 / 같은 클래스의 인스턴스들은 구조는 같지만 각자의 값을 가짐

객체지향 프로그래밍

다양한 용어 정리

- attribute = data = member variable = state = field → 객체가 가진 데이터
- behavior = operation = member function = method → 객체가 하는 일
- 클래스(class) = 개념(concept) = 타입(type) → 설계 · 서술
- 객체(object) = 실체(instance) = 변수(variable) → 실제 존재

클래스 정의 – class Dog (파이썬)

```
class Dog:
    def __init__(self):
        self.x = 0          # attribute
        self.y = 0
    def move(self, a, b):   # behavior
        self.x = a; self.y = b
    def bark(self): ...
    def run(self): ...
```

클래스는 설계일 뿐, 아직 실행되는 것은 없음

객체 생성과 사용

```
d = Dog()           # 오브젝트 생성
d.move(10, 20)      # 메서드 호출
d.bark()

e = Dog()           # 또 다른 오브젝트
print(d.x, e.x)     # 값은 서로 따로
```

객체 d 가 만들어질 때 메모리에 올라감

객체지향 프로그래밍

예제 코드 – 클래스 · 인스턴스 · 속성 찾아보기

- class 뒤의 이름 = 클래스, () 로 만든 변수 = 인스턴스, self.OO = 속성

예제 1 · 선거 후보자

```
class Candidate:          ← 클래스
    def __init__(self, name, party):
        self.name = name   ← 속성
        self.party = party
        self.votes = 0
    def introduce(self):
        print(self.name, self.party)

cand1 = Candidate("후보1", "A당") ← 인스턴스
cand2 = Candidate("후보2", "B당")
cand1.introduce()
```

클래스 : Candidate

인스턴스 : cand1, cand2

속성 : name, party, votes

예제 2 · 게임 캐릭터

```
class Character:          ← 클래스
    def __init__(self, job, atk, hp):
        self.job = job     ← 속성
        self.atk = atk
        self.hp = hp
    def attack(self):
        print(self.job, self.atk)

hero = Character("전사", 216, 1608) ← 인스턴스
mage = Character("마법사", 305, 990)
hero.attack()
```

클래스 : Character

인스턴스 : hero, mage

속성 : job, atk, hp

객체지향 프로그래밍

생성자와 메서드 – 무엇이 다를까

- 생성자는 객체가 만들어질 때 자동으로 한 번, 메서드는 필요할 때마다 내가 호출

코드로 보기

```
class Character:
    region = "61"
    def __init__(self, job, atk): ← 생성자
        self.job = job
        self.atk = atk
    def attack(self): ← 메서드
        print(self.job, self.atk)

hero = Character("전사", 216) ← 자동 실행
hero.attack() ← 직접 호출
hero.attack() ← 또 호출 가능
```

정의할 때 — 클래스 안

```
def __init__(self, job, atk): self 필수
def attack(self): self 필수
```

무엇이 다른가

구분	생성자 (__init__)	메서드 (def)
이름	__init__ 로 고정	자유롭게 정한다
호출	객체 만들 때 자동	필요할 때 직접
횟수	객체마다 한 번	여러 번 가능
역할	속성 초기값 준비	기능 실행
반환	값을 돌려주지 않음	값을 돌려줄 수 있음

호출할 때 — 클래스 밖

```
hero = Character("전사", 216) __init__ 자동
hero.attack() self 생략
```

객체지향 프로그래밍

메서드 – 캐릭터 정보 조회하기 · 객체에게 일을 시키는 방법

- 메서드는 자기 속성(self.OO)을 사용해 일을 처리하는 함수 — 객체.메서드() 로 호출

클래스 안 – 메서드 정의

```
class Character:
    def __init__(self, job, atk, hp):
        self.job = job
        self.atk = atk
        self.hp = hp
    def show_info(self):          ← 조회
        print(self.job, self.atk,
              self.hp)
    def take_damage(self, dmg):  ← 변경
        self.hp -= dmg
    def is_alive(self):          ← 판단
        return self.hp > 0
```

클래스 밖 – 호출과 결과

```
hero = Character("전사", 216, 1608)
hero.show_info()      # 전사 216 1608
hero.take_damage(600)
print(hero.is_alive()) # True
```

조회형 : 속성 값을 보여 준다 (show_info)

변경형 : 속성 값을 바꾼다 (take_damage)

판단형 : 결과를 돌려준다 (is_alive → return)

속성은 객체가 '가진 것', 메서드는 객체가 '할 수 있는 일'

객체지향 프로그래밍

객체지향 키워드 – 강아지 프로그램 하나로 끝까지 따라가기

- 예제는 하나 — 강아지(Dog) 프로그램으로 네 가지 특징 보기

- 클래스 Dog → 인스턴스(오브젝트) my_dog → 푸들 · 불독으로 넓혀 가며 같은 클래스 사용

① 클래스 & 오브젝트

강아지 클래스 → 내 강아지

- 클래스 = 공통 특징의 서술
- 오브젝트 = 인스턴스 = 실제 한 마리

② 캡슐화

내 강아지 안에 담긴 것

- 이름 · 나이는 안에 감춘다
- 짖기 · 달리기만 공개한다

③ 상속성

강아지 → 푸들

- 강아지 특징을 물려받는다
- 다른 점만 새로 쓴다

④ 다형성

“짖어!” 한마디

- 강아지는 멍멍, 푸들은 왈왈
- 같은 명령, 다른 소리

강아지 클래스로 내 강아지를 만들고(①), 속은 감추고(②), 푸들에게 물려주고(③), 같은 “짖어!”에 다르게 답한다(④)

객체지향 프로그래밍

캡슐화 ① 묶기 — 데이터와 데이터를 다루는 기능을 한 상자에

- 오브젝트 = 데이터 + 데이터를 다루는 기능 + 나만의 정체성
 - 값과 그 값을 다루는 함수를 한 상자(클래스)에 함께 담는 것 = 캡슐화

오브젝트 =

data
속성 · 상태값

+

operation on data
데이터를 다루는 기능(메서드)

+

Identity
나만의 정체성 · 구별

class Dog – 한 상자에 함께 담기 (파이썬)

```
class Dog:
    def __init__(self):
        self.x = 0; self.y = 0      ← data
    def move(self, a, b):           ← operation
        self.x = a; self.y = b
    def bark(self):
        print("멍멍!")
    def run(self): ...              묶음 = 캡슐화
```

묶으면 무엇이 좋을까?

- 값과 기능이 한곳에 있어 찾기 쉽다
- 외부에서 값을 마음대로 바꿀 수 없다 → 실수 방지
- 객체마다 자기 값을 따로 가진다 (identity)
- 상자만 골라 다른 프로그램에 재사용할 수 있다

객체지향 프로그래밍

캡슐화 ② 감추기 — 직접 접근 차단, 정해진 메서드로만 사용

- 객체 내부의 메소드와 데이터를 외부에서 사용할 수 없도록 감싸서 객체를 독립적인 프로그램 부품으로 만드는 것, 지정한 인터페이스를 통해서만 외부와 연결



객체지향 프로그래밍

캡슐화 파이썬 구현 — class 키워드와 이름 앞 _ 기호 관례

- class 키워드로 클래스를 만들고, 캡슐화된 클래스를 조합해 프로그램을 구성
 - 파이썬은 접근 제어자 키워드가 없고, 이름 앞의 밑줄(_ , __)로 공개 범위를 표현한다

파이썬 코드로 보기

```
class Dog:
    def __init__(self, name):
        self.name = name      ← 공개
        self.__x = 0         ← 감춤
    def move(self, a):
        self.__x = a
    def bark(self):
        print("멍멍!")
```

```
d.__x = 10 # 에러는 만나지만 __x라는 다른 속성 생성
d.move(10) # 올바른 방법
```

이름 규칙 한눈에 보기 (요약)

name - 공개(public) : 어디서나 접근 가능

_name - 내부용(protected) : 관례상 외부 사용 자제

__name - 감춤(private) : 이름 변형으로 외부 접근 차단

파이썬은 강제가 아닌 '약속(관례)'으로 캡슐화를 표현함 — 접근 허용 범위와 @property 는 뒤 '파이썬의 캡슐화 구현' 장에서 자세히

객체지향 프로그래밍

캡슐화 파이썬 구현

• 접근 제어자에 따른 접근 허용 범위

1. public (공개)

누구나 어디서나 자유롭게 접근 가능

```
class Dog:
    def __init__(self):
        self.name = "바둑이"
        self.bark_sound = "멍멍!"

    def bark(self):
        print(self.bark_sound)
```



사용 예

```
d = Dog()
d.name          # 접근 가능 ✓
d.bark_sound    # 접근 가능 ✓
d.bark()        # 접근 가능 ✓
```

모두
사용할 수 있어요!



2. protected (보호)

클래스 내부와 상속된 클래스에서 접근 가능
(외부에서 접근은 가능하지만 권장하지 않음)

```
class Dog:
    def __init__(self):
        self._age = 3      # 보호된 속성
        self._weight = 10.5 # 보호된 속성

    def _show_info(self): # 보호된 메서드
        print(f"나이: {self._age}, 몸무게: {self._weight}kg")
```



사용 예

```
d = Dog()
d._age          # 접근은 가능하지만 권장하지 않음 ⚠
d._show_info()  # 접근은 가능하지만 권장하지 않음 ⚠
```

```
class MyDog(Dog):
    def print_age(self):
        print(self._age) # 상속된 클래스에서는 OK ✓
```

직접 접근은
가능하지만,
권장하지 않아요!



3. private (비공개)

클래스 내부에서만 접근 가능
(외부에서는 직접 접근 불가)

```
class Dog:
    def __init__(self):
        self.__owner = "주인" # 비공개 속성
        self.__secret = "비밀정보" # 비공개 속성

    def __do_secret(self): # 비공개 메서드
        print("이것은 비밀이에요!")
```



사용 예

```
d = Dog()
d.__owner        # 접근 불가 ✗
d.__do_secret()  # 접근 불가 ✗

(이름 망글링을 통한 우회: 가능하지만 권장하지 않음)
d._Dog__owner    # 우회 가능 (비권장) ⚠
```

직접 접근은
불가능해요!



파이썬의 접근 제어는 엄격한 제한이 아니라 '관례(Convention)'입니다.
다른 개발자와의 협업을 위해 규칙을 지키는 것이 중요해요!

✓ 접근 가능

⚠ 접근 가능하지만 권장하지 않음

✗ 접근 불가

객체지향 프로그래밍

상속성(Inheritance) - 한 클래스의 특징을 다른 클래스가 그대로 물려받는 것

- 비슷한 클래스가 필요할 때, 물려받고 달라진 부분만 새로 작성 (특징 = 데이터 + 조작)

- 물려주는 쪽 = 부모(parent) · 상위(super) 클래스 / 물려받는 쪽 = 자식(child) · 하위(sub) 클래스

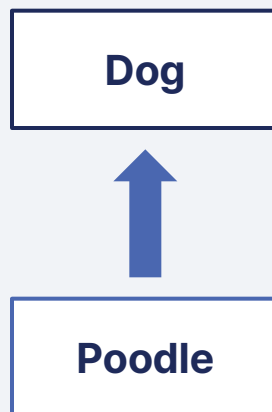
코드로 보기

```
class Dog:
    def __init__(self):
        self.x = 0; self.y = 0
    def move(self, a, b):
        self.x = a; self.y = b
    def bark(self):
        print("멍멍!")
```

```
class Poodle(Dog):    ← 상속
    def bark(self):    ← 새로 정의
        print("왈왈!")
```

```
p = Poodle()
p.move(3, 5)
```

IS-A 관계



Poodle IS-A Dog

Dog 한 마리에서 퍼져 나간 자식 클래스



상속은 복사가 아니라 '물려받기' — 화살표는 상속의 방향이 아니라 "Poodle은 Dog의 한 종류다(IS-A)" 관계 나타냄

객체지향 프로그래밍

상속성(Inheritance) - 한 클래스의 특징을 다른 클래스가 그대로 물려받는 것



객체지향 프로그래밍

다형성 (Polymorphism) - 같은 이름의 명령, 객체마다 다른 동작

- 물려받은 기능을 자식이 자기 방식으로 다시 정의 → 같은 명령에도 결과가 달라짐

- 오버라이딩(overriding) : 물려받은 메서드를 자식 클래스에서 새로 정의하는 것

코드로 보기

```
class Dog:
    def bark(self):
        print("멍멍!")
class Poodle(Dog):
    def bark(self):      ← 재정의
        print("왈왈!")
class Bulldog(Dog):
    def bark(self):      ← 재정의
        print("윙윙!")
```

하나의 호출, 세 가지 결과

bark() 호출



Dog → 멍멍!

Poodle → 왈왈!

Bulldog → 윙윙!

같은 bark() 호출, 강아지마다 다른 소리



같은 명령을 보내도 받는 객체가 누구인지에 따라 알맞게 동작한다 — 여러(poly) 모습(morph)이라는 뜻

객체지향 프로그래밍

다형성 (Polymorphism) - 같은 이름의 명령, 객체마다 다른 동작



객체지향 프로그래밍

파이썬은 무엇이 다를까 – C · 자바와의 비교

- 같은 객체지향이라도 문법과 실행 방식이 다름
- 파이썬 특징 '간결함' & '실행 중 결정'

항목	C · 자바	파이썬
블록 구분	{ } 중괄호로 묶음	들여쓰기(공백 4칸)가 문법
문장 끝	세미콜론 ;	줄바꿈만 하면 끝
변수 타입	int x; 미리 선언	선언 없음 · 실행 중 결정
실행 방식	컴파일 → 실행 파일	인터프리터가 한 줄씩 실행
자료구조	배열을 직접 선언	리스트 · 딕셔너리 내장
시작 지점	main() 함수	if __name__ == "__main__":

객체지향 프로그래밍

파이썬에서 꼭 알아야 할 5가지

- 객체지향 개념은 같지만, 파이썬만의 약속이 있음

코드로 보기

```
class Dog:
    def __init__(self, name):  ← 생성자
        self.name = name
    def bark(self):            ← self 필수
        print(self.name, "멍멍!")
    def __str__(self):         ← 매직 메서드
        return "Dog:" + self.name

d = Dog("초롱이")
print(d)                      # Dog:초롱이
d.age = 3                     ← 속성 추가
```

기억할 다섯 가지

self 메서드 첫 매개변수로 직접 쓴다 (자바 this는 생략)

__init__ 생성자 — 객체가 만들어질 때 자동 실행

__str__ 매직 메서드 — 이름이 약속된 특별한 기능

super() 다중 상속 가능 · 부모 호출에 사용

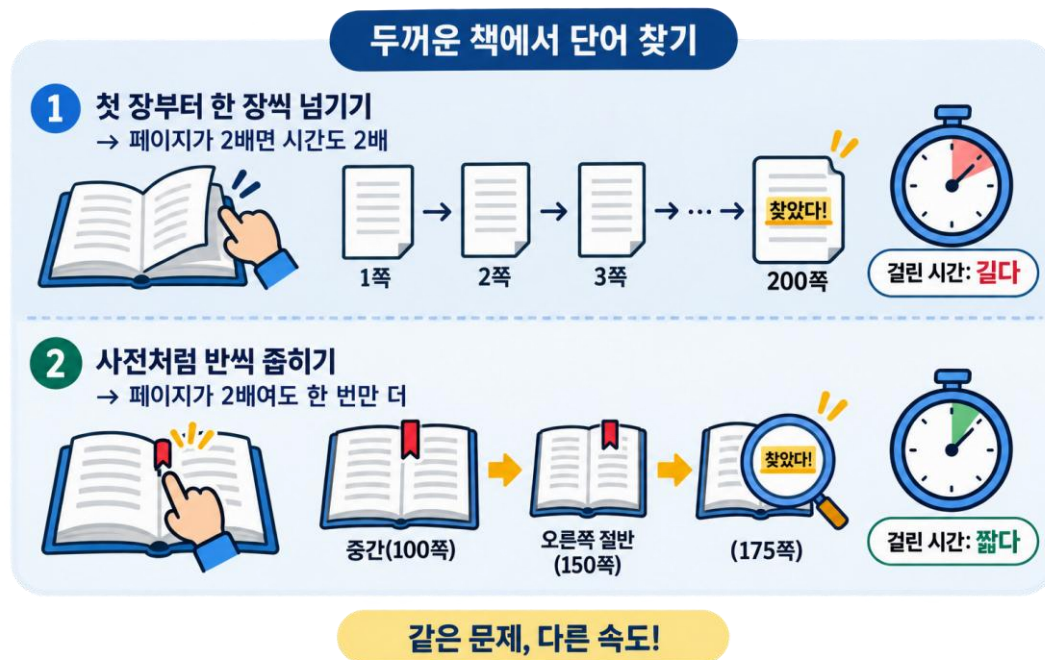
d.age = 3 클래스 밖에서도 속성 추가 가능

주의 : 파이썬은 자유도가 높은 만큼 컴파일러가 막아 주지 않음, 이름 규칙과 들여쓰기 지키기

시간복잡도

시간복잡도란?

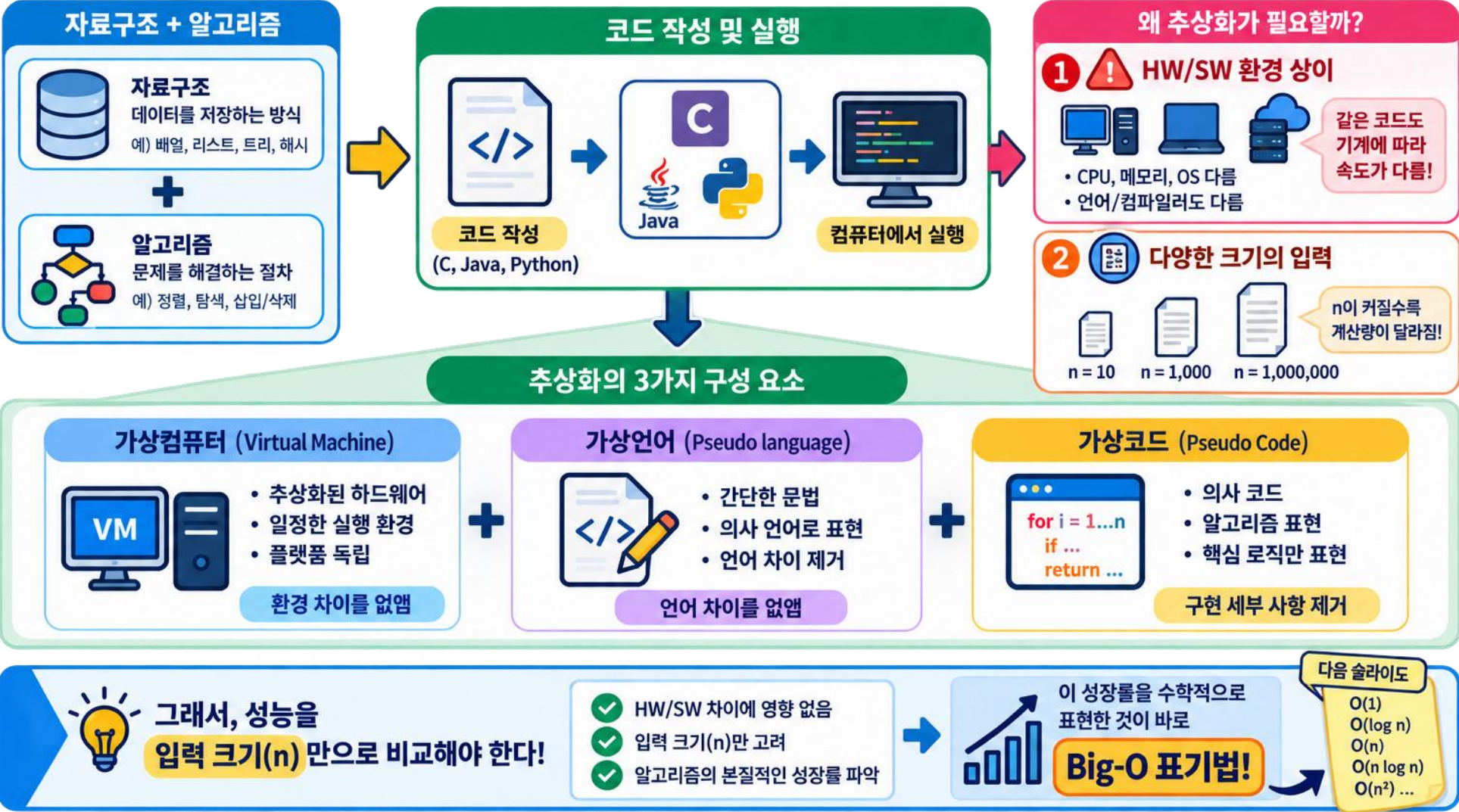
- 입력 데이터가 n 개일 때, 코드가 계산을 몇 번 하는지 세는 것
 - 데이터가 **10개**면 어떤 코드여도 빠름
 - 데이터가 **100만 개**면 방법에 따라 0.01초 vs 3시간으로 갈림
- '초'가 아닌 횟수로 세는 이유? → 컴퓨터 성능마다 걸리는 시간이 달라서 공평한 비교가 어려움





자료구조와 알고리즘: 왜 추상화가 필요한가?

같은 문제를 풀더라도, 실행 환경과 입력 크기에 따라 성능(시간)이 달라진다!



시간복잡도

Big-O 표기법 (Big-O Notation)

■ O는 왜 O인가요?

- O = Order(차수, 규모)의 첫 글자 → "대략 이 정도 규모로 커진다"는 뜻
- $O(n)$ 은 "입력이 2배면 계산도 대략 2배"

■ 두 가지 규칙

- 곱해진 숫자(상수)는 지움 : $2n \rightarrow O(n)$, $n/2 \rightarrow O(n)$
- 가장 크게 자라는 항만 남김 : $n^2 + n \rightarrow O(n^2)$

■ 예제) $3n^2 + 5n + 7 \rightarrow O(n^2)$

- 상수 지우기 $\rightarrow n^2 + n$
- 큰 항만 남기기 $\rightarrow O(n^2)$

왜 최고차항만 볼까?

$n = 1,000,000$ (백만) 일 때

$N^2 = 1\text{조}$ $5n = 500\text{만}$ $7 = 7$

N^2 가 워낙 커서 나머지는 없는 것과 같음

=====

$2n-50$ 과 $8n+100 \rightarrow$ 둘 다 $O(n)$

=====

상수 차이는 더 빠른 컴퓨터로 해결되고,
차수 차이는 안 됨

식이 달라도 최고차항이 같으면 같은 Big-O

$T_1(n) = 2n - 1 \rightarrow O(n)$

$T_2(n) = 4n + 1 \rightarrow O(n)$

$T_3(n) = 1.5n^2 + 1.5n + 1 \rightarrow O(n^2)$

기준 = 최악의 경우 · 운 좋게 빨리 끝나는 경우 X, 끝까지 다 보는 경우 O

시간복잡도

시간복잡도 계산 예제

값 하나만 꺼내기

```
check(arr[0])
```

데이터가 몇 개든 계산은 한 번

$O(1)$

전체 한 번 훑기

```
for i in range(n):
```

```
    check(arr[i])
```

range(n) 은 0 ~ n-1 → 반복 n번

$O(n)$

반복문 두 겹

```
for i in range(n):
```

```
    for j in range(n):
```

```
        check(i, j)
```

바깥 n번 × 안쪽 n번 → $n \times n$ 번

$O(n^2)$

$n = 1,000$ 이라면?

$O(1) \rightarrow 1$ 번 | $O(n) \rightarrow 1,000$ 번 | $O(n^2) \rightarrow 1,000,000$ 번 같은 데이터인데 계산량은 100만 배 차이!

시간복잡도

프로그램 스텝 (Program Step) — 계산 횟수를 코드로 직접 세어 보기

```
def total_sum(n):  
    total = 0  
    STEP = 0  
    for i in range(1, n + 1):  
        total = total + i  
        STEP = STEP + 1  
    return total, STEP  
  
total_sum(100) → (5050, 100)
```

프로그램 스텝이란?

- 실행 시간을 한눈에 알아챌 수 있게 만든 **계산 단위**
- 초가 아니라 **횟수**라서 컴퓨터가 달라도 같음
- 한 줄이 **스텝 1번**, 반복되면 반복만큼 늘어남

결과 읽기

n = 100 일 때 스텝 **100번**

즉 $T(n) = n \rightarrow O(n)$

Quiz: 여기서 반복문을 한 겹 더 넣으면 **STEP**은 몇 번이 될까요?

시간복잡도

■ Big-O 증명 — $f(n) = 1000n + 5000$ 이 $O(n)$ 임을 정의로 확인하기

Big-O 의 정의

- 모든 $n \geq n^*$ 에서 $f(n) \leq c \cdot g(n)$ 인 c 와 n^* 이 존재하면 $f(n) = O(g(n))$

증명 방법

- c 와 n^* 을 하나만 찾아 보이면 증명 완료
- 상수항을 n 의 배수로 바꿔 묶는 것이 핵심
- c, n^* 은 유일하지 않음 ($c = 10000$ 도 성립)

계수 1000 과 상수 5000 은 n 이 커질수록 영향이 사라진다 — 그래서 $O(n)$ 으로 단순화함

시간복잡도

데이터 하나가 늘면? — $O(n^2)$ 와 $O(2^n)$ 의 차이

n^2 : 한 칸 늘면 더하기

$$(n+1)^2 = n^2 + (2n + 1)$$

$n = 1,000 \rightarrow 2,001$ 번만 더 계산 (+0.2%)

2^n : 한 칸 늘면 곱하기

$$2^{(n+1)} = 2^n \times 2$$

데이터 하나에 계산량이 **2배 (+100%)**

n	n^2	2^n	2^n 걸리는 시간
10	100	1,024	0.00001초
20	400	약 100만	0.01초
30	900	약 10억	10초
40	1,600	약 1조	약 3시간
50	2,500	약 1,000조	약 4개월
60	3,600	약 100경	약 36년

n^2 은 느리지만 쓸 수 있는 코드, 2^n 은 못 쓰는 코드입니다.

매일 2,000원씩 더 받기 vs 매일 2배가 되기 (1원 시작 $\rightarrow$ 30일 뒤 약 10억 원)

시간복잡도

연습문제 : 수행시간이 오래 걸리는 순서대로 나열해 봅시다

$$O(7n \log n + 100)$$

$$O(3^{n+2})$$

$$O(n^{10})$$

$$O(5n!)$$

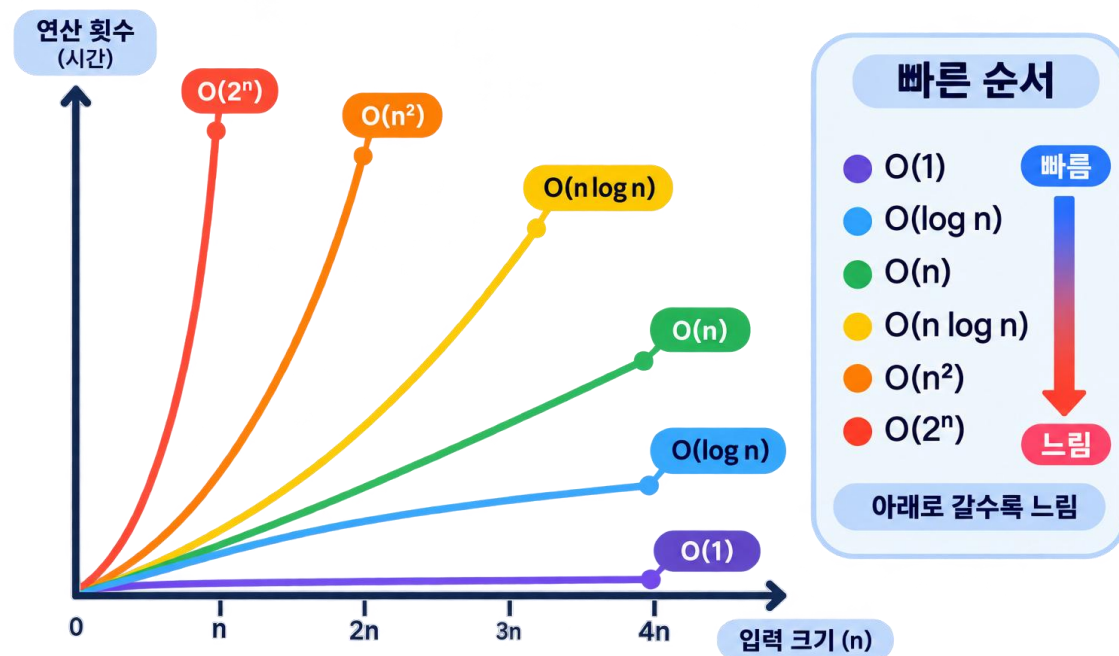
앞에 붙은 숫자(7, 3, 5)와 더해진 상수(+100, +2)보다 n 이 어디에 있는지!

n 이 밑에 있나(n^{10}), 지수에 있나(3^n), 아니면 팩토리얼인가($n!$) — 느린 순서는 $n! > 3^n > n^{10} > n \log n$

시간복잡도

복잡도 비교

복잡도	설명	예시	계산 횟수
$O(1)$	그냥 한 번	arr[0] 꺼내기	1
$O(\log n)$	반씩 줄이기	이진 탐색	약 10
$O(n)$	한 번 훑기	리스트 전체 확인	1,000
$O(n \log n)$	훑기 + 반씩	정렬(sort)	약 10,000
$O(n^2)$	반복문 두 겹	모든 짝 비교	1,000,000
$O(2^n)$	매번 두 배	모든 경우 다 만들기	사실상 불가능



시간복잡도

정렬 알고리즘 두 가지 — 공 1~5를 오름차순으로

① 선택 정렬 (Selection Sort)

- 남아 있는 공 중에서 가장 작은 공을 찾는다
- 찾은 공을 맨 앞으로 보낸다
- 나머지 공들에 대해 같은 일을 반복한다

가장 작은 값을 '고른다' → 매번 전부 확인



② 병합 정렬 (Merge Sort)

- 공 그룹을 반으로 계속 쪼갬다 (1개가 될 때까지)
- 두 그룹을 앞에서부터 비교하며 순서대로 합친다
- 합치기를 끝까지 반복하면 정렬 완료

반씩 쪼개니 단계가 $\log n$ 번



결과는 동일(1 2 3 4 5) · 차이는 "비교 횟수"

공 5개 → 차이 거의 없음 · 공 100만 개 → 결과 완전히 다름

시간복잡도

① 선택 정렬 — 코드와 실행 연산 횟수

```
def selection_sort(a):  
    n = len(a)  
    for i in range(n - 1):  
        m = i  
        for j in range(i+1, n):  
            if a[j] < a[m]: ← 비교  
                m = j  
        a[i], a[m] = a[m], a[i] ← 교환  
    return a
```

비교 횟수를 세어보면

- 1번째: 4번 비교 → 2번째: 3번 → 3번째: 2번 → 4번째: 1번
- $4 + 3 + 2 + 1 = 10$ 번

일반화하면

$$T(n) = n(n-1)/2 = (n^2 - n)/2$$

- 상수 1/2 지우고, 큰 항 n^2 만 남기면

$$\rightarrow O(n^2)$$

공 5개 → 10번 · 공 1,000개 → 약 50만 번 · n 커질수록 제곱으로 증가

시간복잡도

② 병합 정렬 — 코드와 실행 시간 (공 5개, 한 연산 = 1ns 가정)

쪼개는 단계는 몇 번?

- 5개 $\rightarrow$ 2·3개 $\rightarrow$ 1·1·2개 $\rightarrow$ 1개 : 약 3단계 ($= \log_2 5$)

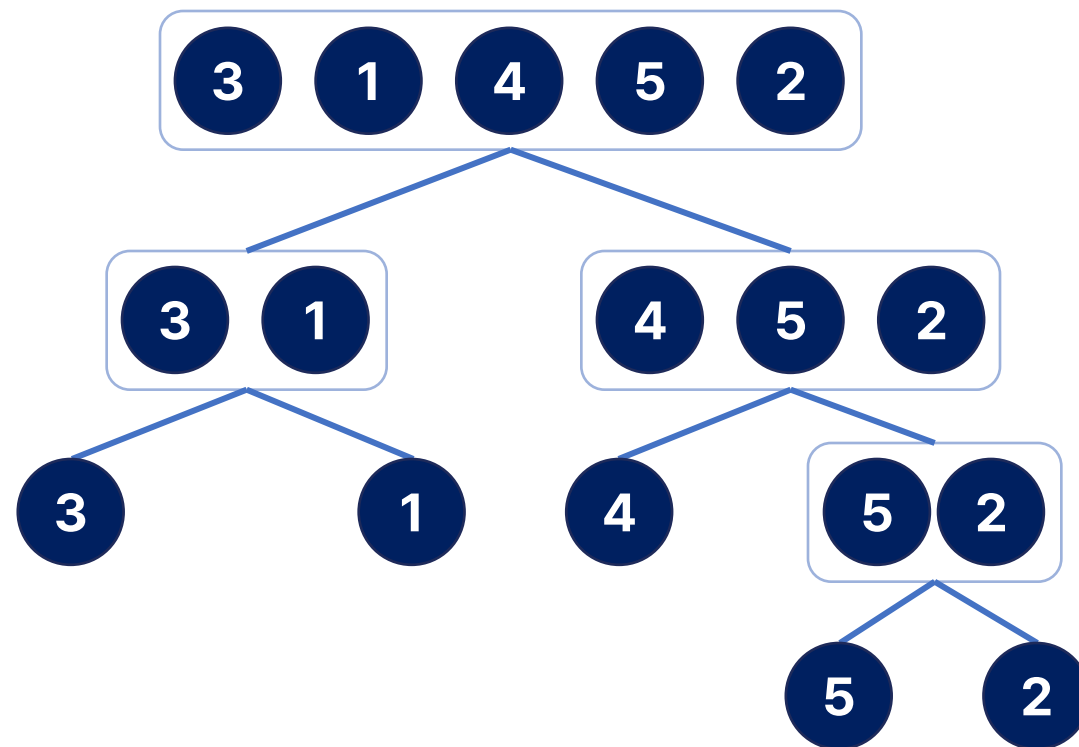
- 각 단계에서 합칠 때 최대 5번 비교

$3 \times 5 \approx 12 \sim 15$ 번 비교 $\approx 15\text{ns}$

일반화하면

$$T(n) = n \times \log_2 n$$

$\rightarrow O(n \log n)$



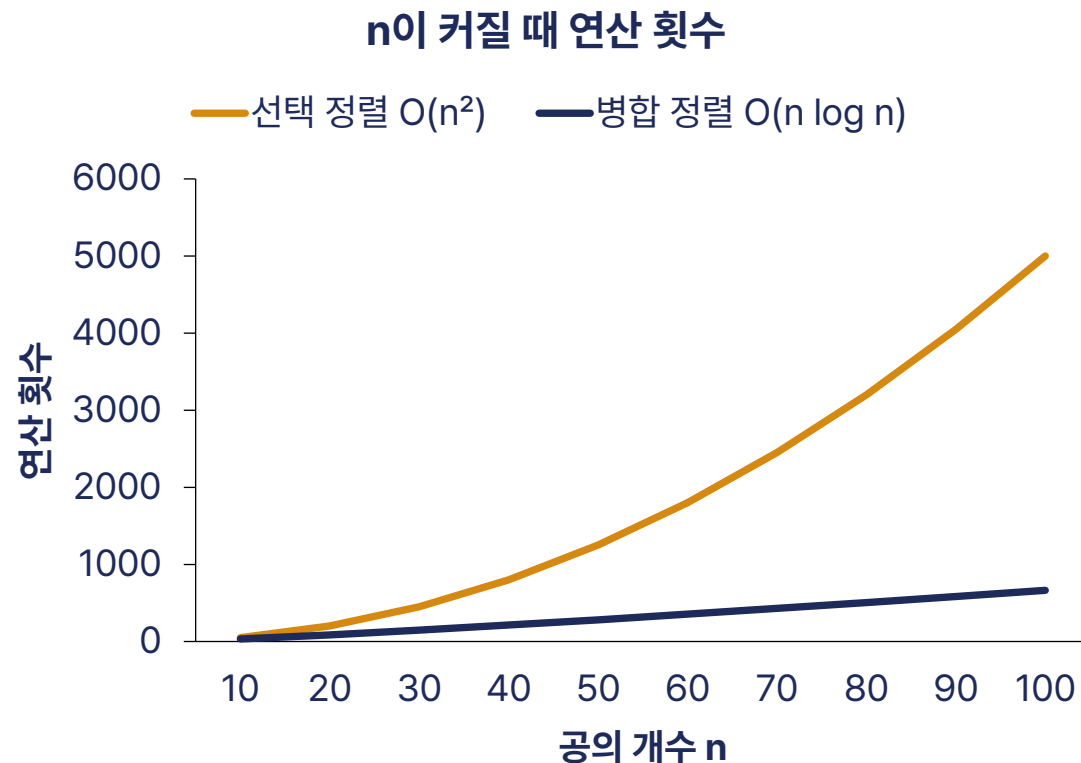
공 5개 $\rightarrow$ 선택 정렬(10번)과 큰 차이 없음

$n = 1,000 \rightarrow$ 선택 약 50만 번 vs 병합 약 1만 번 (약 50배)

시간복잡도

두 알고리즘 비교— $O(n^2)$ vs $O(n \log n)$

공의 개수 n	선택 정렬 ($n^2/2$)	병합 정렬 ($n \log_2 n$)	차이
5개	10번	약 12번	거의 없음
1,000개	약 500,000번	약 10,000번	50배
1,000,000개	약 5,000억 번	약 2,000만 번	25,000배



공 5개는 거의 같지만, 100만 개에서는 25,000배로 벌어짐

시간복잡도

자료구조 ① 특정 값 찾기 : 배열 $O(n)$ → 해시테이블 $O(1)$

배열 순차 탐색 → $O(n)$

```
arr = [34, 7, 23, 32, 5, 62]
```

```
print(len(arr)) # 6
```

```
def find(target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

최악 n번 비교



시간복잡도

자료구조 ① 특정 값 찾기 : 배열 $O(n)$ → 해시테이블 $O(1)$

해시테이블(set/dict) → $O(1)$

```
table = set()    내부가 해시테이블
table.add(34)
table.add(7)    ...
```

```
def find(target):
    return target in table    비교 1번
```



왜 빠른가 : 해시 함수로 "몇 번째 칸"을 계산 → 앞에서부터 훑기 X

비유 : 전화번호부 한 장씩 = $O(n)$ ↔ 사서가 청구기호로 바로 = $O(1)$

시간복잡도

자료구조 ② 정렬된 데이터에서 값 찾기 : 배열 $O(n)$ → 이진탐색 $O(\log n)$

개선 전 : 정렬 안 된 리스트 → $O(n)$

arr = [62, 5, 32, 23, 7, 34] 정렬 X

```
def find(target):  
    for i in range(len(arr)):  
        if arr[i] == target:  
            return i  
    return -1
```

최악 n번

개선 후 : 정렬된 리스트 + 이진탐색 → $O(\log n)$

arr = [5, 7, 23, 32, 34, 62] 정렬 O

```
low, high = 0, len(arr) - 1  
while low <= high:  
    mid = (low + high) // 2  
    if arr[mid] == target: return mid  
    elif arr[mid] < target: 오른쪽 절반만  
        low = mid + 1  
    else: high = mid - 1 왼쪽 절반만
```

왜 빠르냐 : 비교 1번 = 범위 절반 → 최대 $\log_2 n$ 번

1,000개 → 순차 1,000번 vs 이진 약 10번 · 100만 개 → 약 20번 · 전제 : 정렬 필요

시간복잡도

자료구조 ③ 맨 앞에 데이터 삽입 : 배열 $O(n)$ → 연결리스트 $O(1)$

개선 전 : 리스트 맨 앞에 삽입 → $O(n)$

```
arr = [1, 2, 3, 4]
arr.insert(0, 0) ← 한 줄처럼 보임
# 내부 동작!!!
arr = [1, 2, 3, 4]
n = len(arr)
arr.append(None) ← 칸 확보
for i in range(n, 0, -1):
    arr[i] = arr[i-1] ← 이동 n번
arr[0] = 0
```

개선 후 : deque 맨 앞에 삽입 → $O(1)$

```
from collections import deque

dq = deque([1, 2, 3, 4])

dq.appendleft(0) 연산 1번

# 양끝 연결만 갱신
# 기존 원소 이동 0회
```

왜 빠른가 : 양끝 연결만 갱신 → 기존 원소 이동 0회

단, 인덱스 접근은 $O(n)$ · 자료구조 = 무엇이 빨라지고 무엇이 느려지는가의 교환

시간복잡도

연습문제 : 네 코드의 시간복잡도를 각각 적어 보세요

A 리스트의 첫 번째 값 꺼내기

```
def first(a):  
    return a[0]
```

시간복잡도 $O(\text{_____})$

B 전체 값을 모두 더하기

```
total = 0  
for i in range(n):  
    total = total + a[i]
```

시간복잡도 $O(\text{_____})$

C 모든 두 값의 쌍을 비교하기

```
for i in range(n):  
    for j in range(n):  
        compare(a[i], a[j])
```

시간복잡도 $O(\text{_____})$

D 범위를 절반씩 줄여 찾기

```
while low <= high:  
    mid = (low + high) // 2  
    ... low/high 를 절반으로 ...
```

시간복잡도 $O(\text{_____})$

시간복잡도

자료구조와 Big-O

- 같은 작업(삽입, 삭제, 탐색)이어도 어떤 자료구조를 쓰느냐에 따라 소요 시간이 상이함
- n 이 커져도 연산 횟수가 변하지 않는가?

자료구조	특정 원소 탐색	맨 앞에 삽입	인덱스로 접근
배열	$O(n)$	$O(n)$ (다 밀어야 함)	$O(1)$ ← 빠름
연결리스트	$O(n)$	$O(1)$ ← 빠름	$O(n)$ (처음부터 따라가야 함)
해시테이블	$O(1)$ (평균)	—	—
정렬된 이진탐색트리	$O(\log n)$	$O(\log n)$	—

시간복잡도

pop() vs pop(0) – 왜 복잡도가 다른가 (C 와 비교)

arr.pop() 맨 끝에서 꺼내기 → O(1)

파이썬

```
arr = [1, 2, 3, 4, 5]
arr.pop()    # 맨 끝 원소(5) 제거
# arr = [1, 2, 3, 4]
```

C 로 직접 구현하면

```
int arr[] = {1,2,3,4,5}; int n = 5;
int last = arr[n-1]; // 값만 꺼냄
n--;                // 크기만 줄임
// 원소를 옮기는 연산이 없음
```



마지막 칸만 비우면 끝 – 이동 0회

파이썬도 내부적으로 길이만 1 줄이는 것과 같다.

뒤에 아무도 없으니 옮길 원소가 없다 → O(1)

arr.pop(0) 맨 앞에서 꺼내기 → O(n)

파이썬

```
arr = [1, 2, 3, 4, 5]
arr.pop(0)  # 맨 앞 원소(1) 제거
# arr = [2, 3, 4, 5]
```

C 로 직접 구현하면

```
for (int i = 0; i < n-1; i++)
    arr[i] = arr[i+1]; // n-1번 이동
n--;
// 파이썬 pop(0) 이 내부에서 하는 일과 동일
```



↓ 맨 앞(1)을 빼면



남은 2,3,4,5 를 전부 한 칸씩 앞으로 밀어야 한다 (n-1회).

남은 원소를 전부 한 칸씩 앞으로 이동 → O(n)

시간복잡도

전체 정리 — 자료구조를 바꾸면 복잡도가 바뀜

문제	기존 자료구조	복잡도	개선 자료구조	개선 후	왜 빨라지는가
값 찾기	배열 (순차 탐색)	$O(n)$	해시테이블	$O(1)$	계산으로 위치를 즉시 파악
값 찾기	정렬 안 된 배열	$O(n)$	정렬된 배열 / BST	$O(\log n)$	비교마다 범위를 절반으로
맨 앞 삽입	배열	$O(n)$	연결리스트	$O(1)$	포인터만 바꾸면 됨 (이동 X)

핵심 : 가장 중요한 항만 남기고, 최대한 단순하게
상수는 영향이 크지 않음 · **n이 한없이 커질 때**를 기준으로 비교함

시간복잡도

부록 : 자료구조별 Big-O 비교 — 평균과 최악의 경우

자료구조	탐색 (평균)	삽입 (평균)	삭제 (평균)	탐색 (최악)	삽입 (최악)	삭제 (최악)
정렬된 배열	$O(\log n)$	$O(n)$	$O(n)$	$O(\log n)$	$O(n)$	$O(n)$
배열	$O(n)$	N/A	N/A	$O(n)$	N/A	N/A
연결리스트	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(1)$	$O(1)$
이중 연결리스트	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(1)$	$O(1)$
스택	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(1)$	$O(1)$
해시테이블	$O(1)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
이진탐색트리	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$
B-트리	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
레드-블랙 트리	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
AVL 트리	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$

THANK YOU

Email | jieonkim@hallym.ac.kr

Tel. | 033-248-2353